

Sistemas Operacionais

Utilize para os estudos os livros apresentados na referência bibliográfica da disciplina. Há um e-book (autor Maziero) compartilhado no moodle que pode ser usado para estudo e, neste livro, o assunto da avaliação compreende os capítulos de 2 a 13.

1 Fundamentos, história e estrutura do SO Aulas 2-3

- ☐ SO como máquina estendida (abstrações) × gerenciador de recursos.
- ☐ Modo núcleo × modo usuário, interrupções e chamadas de sistema (programa → API → kernel).
- ☐ Gerações: lote → multiprogramação → tempo compartilhado → PCs/SOs modernos.
- ☐ Arquiteturas: monolítica, camadas, micronúcleo, híbrida e máquinas virtuais.

💡 **Refleta:** Por que uma falha em um driver pode derrubar um SO monolítico, mas não um micronúcleo? O que o micronúcleo “paga” por essa robustez?

💡 Que problema da época a multiprogramação resolveu — e que novos problemas (proteção, concorrência) ela criou?

2 Processos Aula 4

- ☐ Programa (passivo) × processo (ativo); código, dados, pilha, heap e contexto.
- ☐ PCB e troca de contexto: o que é salvo/restaurado e por que isso tem custo.
- ☐ Diagrama de estados (novo, pronto, executando, bloqueado, terminado) e suas transições.
- ☐ `fork()`, `exec()`, `wait()`; processo zumbi e órfão.

💡 **Refleta:** Por que não existe a transição bloqueado → executando? O que aconteceria se existisse?

💡 Quantas linhas imprime um laço com 3 chamadas a `fork()`? Desenhe a árvore antes de responder.

3 Threads Aula 5

- ☐ O que a thread tem de próprio (PC, registradores, pilha) × o que compartilha (código, dados globais, heap, arquivos).
- ☐ Threads de usuário × de núcleo; modelos N:1, 1:1 e N:M (vantagens e limitações).
- ☐ `pthread_create` / `pthread_join` e o risco do compartilhamento sem sincronização.

💡 **Refleta:** Um servidor web deve usar processos ou threads? E se, no modelo N:1, uma thread fizer uma E/S bloqueante?

4 Escalonamento de CPU Aula 9

- ☐ Quando escalonar; preemptivo × cooperativo;.
- ☐ Métricas: tempo de espera, de execução (turnaround) e de resposta; objetivos por tipo de sistema (lote, interativo, tempo real).
- ☐ Algoritmos: FCFS, SJF/SRTF, Round-Robin (quantum), prioridades (com envelhecimento), loteria, múltiplas filas.
- ☐ Saber montar o diagrama de Gantt e calcular tempos médios à mão.

💡 **Refleta:** SJF é ótimo para tempo médio de espera — por que quase não é usado puro na prática? Que risco ele traz?

💡 No Round-Robin, o que acontece com quantum muito grande? E muito pequeno? Como a troca de contexto entra na conta?

5 Concorrência e exclusão mútua Aula 7

- ☐ Condição de disputa (erro dinâmico, depende da ordem) e região crítica; condições de Bernstein.
- ☐ Requisitos de uma boa solução: exclusão mútua, progresso, espera limitada, independência de velocidade.
- ☐ Evolução das soluções: inibir interrupções → variável de trava → alternância estrita → Peterson → instruções atômicas (TSL) → sleep/wakeup → semáforos.
- ☐ Espera ocupada, inanição e inversão de prioridade.

💡 **Refleta:** Por que `saldo = saldo + v` não é atômico? Mostre uma intercalação que perde um depósito.

💡 Para cada solução da evolução acima: qual defeito dela motivou a próxima? (Esse encadeamento é o coração do tema.)

6 Semáforos e monitores Aulas 7-8

- ☐ **down/up** (P/V); semáforo contador × mutex (binário); por que as operações precisam ser atômicas.
- ☐ Produtor-consumidor com três semáforos: mutex, vagas (vazio) e itens (cheio).
- ☐ Monitores (Hoare, Brinch Hansen): encapsulamento, exclusão implícita, variáveis de condição (wait/signal).

 **Refleta:** No produtor, o que acontece se você inverter **down (vazio)** e **down (mutex)**? Simule com o buffer cheio.

 Que tipos de erro com semáforos o monitor torna impossíveis — e quais ainda podem acontecer?

7 Problemas clássicos de coordenação Aula 10

- ☐ Produtores/consumidores (buffer limitado): exclusão mútua + suspensão com buffer cheio/vazio.
- ☐ Leitores/escritores: leituras em paralelo, escrita exclusiva; prioridade a leitores e inanição de escritores.
- ☐ Jantar dos filósofos: por que a solução ingênua trava e as formas de corrigir (assimetria, limitar a N-1, pegar os dois garfos de uma vez).

 **Refleta:** Reconheça o problema clássico por trás de: fila de impressão, banco de dados com consultas e atualizações, transferências entre contas.

8 Deadlocks (impasses) Aula 11

- ☐ As 4 condições de Coffman: exclusão mútua, posse e espera, não preempção, espera circular (necessárias, não suficientes).
- ☐ Grafo de alocação de recursos: ciclo com 1 instância por recurso = impasse; com várias instâncias, não necessariamente.
- ☐ Estratégias: avestruz, prevenção (quebrar uma condição), impedimento (estado seguro, algoritmo do banqueiro), detecção e recuperação (abortar, rollback, preempção).
- ☐ Diferença entre deadlock, inanição (starvation) e livelock.

 **Refleta:** Duas tarefas transferem dinheiro entre as contas A e B em sentidos opostos e travam. Proponha duas soluções que quebrem condições diferentes e compare custos.

 Por que a maioria dos SOs de uso geral adota o “algoritmo do avestruz”? Em que tipo de sistema isso seria inaceitável?

 **Checagem final — você está pronto se consegue, sem consultar:**

simular FCFS/SJF/RR em um Gantt · apontar a região crítica em um trecho de código · escrever produtor-consumidor com semáforos · desenhar um grafo de alocação, dizer se há impasse e propor como sair dele · explicar por que cada solução de exclusão mútua substituiu a anterior.